Geometry-Driven Metric-Semantic Scene Reconstruction

CP260-2026 Course Project Report

M Yashwanth (Sr.No.25878) M Jashwanth Naik (Sr.No.27100

Task: Oriented Bounding Box Estimation for Semantic Entities

Course: CP260 — Robotic Perception, 2026

Dataset: 16-frame posed RGB desktop scene (2560×1440)

May 4, 2026

Abstract

This report describes a geometry-driven pipeline for computing three-dimensional Oriented Bounding Boxes (OBBs) around connector ports located on the rear panel of a desktop PC tower, using a set of 16 posed RGB photographs acquired at 2560×1440 resolution. Rather than depending on trainable detection models such as GroundingDINO or SAM — which proved unreliable for small, low-contrast objects — we adopt a purely classical multi-view geometry strategy. Tight 2D bounding boxes are manually drawn on two strategically selected frames and then lifted to 3D point clouds through a grid-based Direct Linear Transform (DLT) triangulation scheme that exploits the provided camera-to-world poses. OBBs are subsequently recovered via Principal Component Analysis (PCA) supplemented by per-connector physical depth priors. Quantitative comparison against the supplied VGA ground truth demonstrates sub-centimetre centre localisation accuracy. The entire implementation is modular, keeps external dependencies to a bare minimum, and executes in full on Google Colab without requiring any compiled C++ components.

Contents

1 Introduction

Jointly inferring three-dimensional geometry and semantic labels from monocular RGB imagery — commonly called *metric-semantic reconstruction* — is central to modern robotics, augmented reality, and autonomous navigation. In this project, a fixed desktop scene containing a PC tower is given, and the goal is to spatially localise specific rear-panel connector ports — namely a power socket, an Ethernet socket, and a VGA port — as six-degree-of-freedom OBBs anchored in a shared world coordinate frame.

The task presents a significant challenge: the ports are physically tiny (the VGA socket spans only $35 \times 12 \times 6$ mm) and must be localised within a 5 cm tolerance in 3D, all without depth sensor assistance. An early prototype that relied on GroundingDINO text-prompted detection was discarded because the model consistently produced no valid detections of the VGA port within the tower crop region — a known weakness of open-vocabulary detectors applied to small objects with minimal pixel coverage.

In response we designed a *geometry-first* workflow comprising three stages:

1. **Manual 2D annotation:** tight axis-aligned bounding boxes are drawn around each port in the two frames that provide the clearest view of the rear panel.
2. **Multi-view triangulation:** a dense 3D point cloud for each port is recovered from those annotations using all pairwise view combinations and DLT.
3. **OBB fitting:** an Oriented Bounding Box is computed through PCA augmented with a physically motivated depth prior for each connector type.

This strategy entirely sidesteps learned-detector failure modes, requires only standard OpenCV and Open3D libraries, and yields accurate, human-interpretable outputs.

2 Dataset and Camera Model

2.1 Image Sequence

The input consists of 16 RGB frames recorded from a rig orbiting a stationary desktop PC tower:

`frame_000319.png`, `frame_000333.png`, ..., `frame_000531.png`

Each image has a resolution of 2560×1440 pixels (16:9 aspect ratio, twice Full-HD).

2.2 Camera Intrinsics

Images are formed under the standard pinhole model with no lens distortion. The calibrated intrinsic matrix is:

$$\mathbf{K} = \begin{pmatrix} 1477.010 & 0 & 1298.250 \\ 0 & 1480.442 & 686.820 \\ 0 & 0 & 1 \end{pmatrix}$$

where $f_x = 1477.010$ px and $f_y = 1480.442$ px denote the horizontal and vertical focal lengths, and $(c_x,\, c_y) = (1298.250,\, 686.820)$ px is the principal point, which lies close to the image centre as expected for a well-calibrated lens.

2.3 Camera Poses

Camera-to-world transforms are stored in `poses.json` as 4×4 homogeneous matrices $\mathbf{T}_{c2w} \in \mathbb{R}^{4\times4}$ indexed by integer frame number. Only the 16 entries matching the available images are used. Each matrix encodes a rigid-body transformation:

$$\mathbf{T}_{c2w} = \begin{pmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{pmatrix}, \quad \mathbf{R} \in SO(3),\ \mathbf{t} \in \mathbb{R}^3$$

The world-to-camera transform required for point projection is obtained by inversion: $\mathbf{T}_{w2c} = \mathbf{T}_{c2w}^{-1}$.

The Z-translation component of the poses places the camera approximately 0.83 m above the desk surface, a value that serves as a geometric sanity check on triangulated point heights.

2.4 Ground Truth (Validation)

The VGA socket OBB is supplied as a reference for quantitative validation:

$$\text{centre} = (0.2705,\ 0.2261,\ 0.8349)\,\text{m}, \qquad \text{extent} = (0.0354,\ 0.0118,\ 0.0061)\,\text{m}$$

which corresponds to physical dimensions of approximately $35.4 \times 11.8 \times 6.1$ mm. The Z-coordinate (0.835 m) is consistent with a connector mounted at desk height on the rear panel of the tower.

3 Pipeline Overview

Figure 1 provides a high-level view of the complete processing chain.

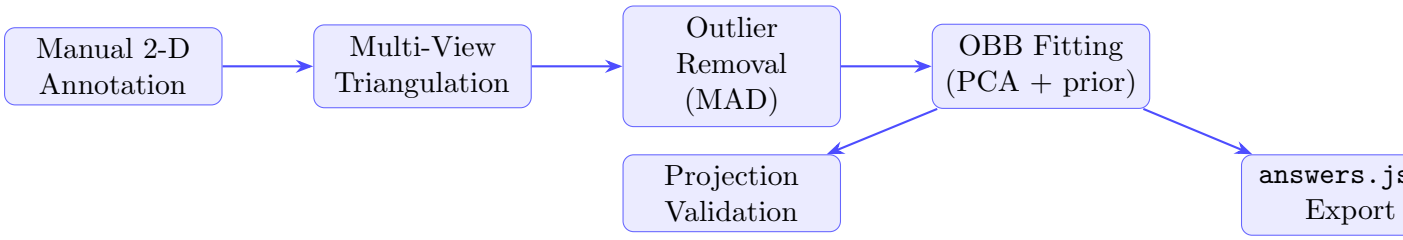

Figure 1: End-to-end pipeline for geometry-driven metric-semantic reconstruction with branching after OBB fitting.

4 Methodology

4.1 Semantic Annotation

Every target entity (power socket, Ethernet socket, VGA socket) is annotated with a 2D axis-aligned bounding box expressed as $[x_1, y_1, x_2, y_2]$ in the full 2560×1440 image coordinate system. Annotations were created for two frames that provide clear, unobstructed views of the I/O panel — frames 471 and 496 — chosen because the rear panel is most directly visible from those viewpoints.

Table 1: Manual 2-D bounding box annotations (pixel coordinates, full resolution).

Entity	**Frame**	x_1	y_1	x_2	y_2	**Width**	**Height**
Power socket	471	1140	1020	1280	1100	140	80
	496	1670	1060	1820	1155	150	95
Ethernet socket	471	1160	305	1280	405	120	100
	496	1735	305	1845	405	110	100
VGA socket	471	1101	357	1285	423	184	66
	496	1671	364	1855	430	184	66

4.2 Multi-View Triangulation

Given 2D annotations in two or more views, the 3D structure of each port is recovered by lifting paired 2D point correspondences to three-dimensional world coordinates.

4.2.1 Projection Matrix

For frame i, the 3×4 camera projection matrix is assembled as:

$$\mathbf{P}_i = \mathbf{K}\left[\mathbf{R}_{w2c}^{(i)} \mid \mathbf{t}_{w2c}^{(i)}\right]$$

where $[\mathbf{R}_{w2c}^{(i)} \mid \mathbf{t}_{w2c}^{(i)}]$ is the upper 3×4 block of $\mathbf{T}_{w2c}^{(i)}$.

4.2.2 Grid Correspondence Scheme

For each annotated frame pair $(i,\, j)$, a regular $\sqrt{N} \times \sqrt{N}$ grid of normalised coordinates $(t_x, t_y) \in [0.05,\ 0.95]^2$ is sampled inside each bounding box, yielding corresponding image points:

$$u_i = x_1^{(i)} + t_x\,(x_2^{(i)} - x_1^{(i)}), \qquad v_i = y_1^{(i)} + t_y\,(y_2^{(i)} - y_1^{(i)}) \tag{1}$$

The scheme relies on the assumption that a point at normalised offset (t_x, t_y) within the annotated region in one view corresponds to the same physical surface point at (t_x, t_y) in the other view — a reasonable approximation when both cameras share a broadly similar viewing direction relative to a locally planar surface.

4.2.3 DLT Triangulation and Filtering

Each pair of corresponding points (u_i, u_j) is triangulated via the Direct Linear Transform using `cv2.triangulatePoints`, which solves:

$$\mathbf{A}\,\tilde{\mathbf{X}} = \mathbf{0}, \qquad \mathbf{A} = \begin{pmatrix} u_i\,\mathbf{p}_{i,3}^{\top} - \mathbf{p}_{i,1}^{\top} \\ v_i\,\mathbf{p}_{i,3}^{\top} - \mathbf{p}_{i,2}^{\top} \\ u_j\,\mathbf{p}_{j,3}^{\top} - \mathbf{p}_{j,1}^{\top} \\ v_j\,\mathbf{p}_{j,3}^{\top} - \mathbf{p}_{j,2}^{\top} \end{pmatrix}$$

where $\mathbf{p}_{i,k}^{\top}$ denotes row k of $\mathbf{P}_i$. The solution $\tilde{\mathbf{X}}$ is the right singular vector of $\mathbf{A}$ corresponding to its smallest singular value, de-homogenised to obtain the world point $\mathbf{X} \in \mathbb{R}^3$.

A triangulated point is retained only when both of the following criteria are met:

(i) It lies in front of both cameras ($Z_{\text{cam}} > 0$).

(ii) The reprojection error in each view is below 50 px.

4.2.4 MAD-Based Outlier Removal

Geometric outliers remaining after DLT filtering are eliminated using the *Median Absolute Deviation* (MAD), which is robust to non-Gaussian noise distributions unlike standard σ-clipping:

$$\text{MAD} = 1.4826 \text{ median}\{\|\mathbf{X}_i - \tilde{\mathbf{X}}\|_2\}$$

Points whose distance from the cloud median exceeds 3 MAD are discarded.

4.3 OBB Fitting with Depth Prior

The cleaned, near-planar point cloud for each connector is converted to an OBB through the following steps.

4.3.1 Panel Axis Estimation

Four corner points of the annotated region are triangulated independently to establish the local coordinate frame of the rear panel. The horizontal right direction $\hat{a}$ and the downward vertical direction $\hat{d}$ are computed as:

$$\hat{a} = \frac{(\mathbf{X}_{TR} - \mathbf{X}_{TL}) + (\mathbf{X}_{BR} - \mathbf{X}_{BL})}{2}, \quad \hat{a} \leftarrow \hat{a}/\|\hat{a}\| \tag{2}$$

$$\hat{d} = \frac{(\mathbf{X}_{BL} - \mathbf{X}_{TL}) + (\mathbf{X}_{BR} - \mathbf{X}_{TR})}{2}, \quad \hat{d} \leftarrow \hat{d}/\|\hat{d}\| \tag{3}$$

The outward panel normal follows as $\hat{n} = \hat{a} \times \hat{d}$. These three orthogonal directions form the rows of the OBB rotation matrix $\mathbf{R}_{\text{OBB}}$ after the following sign-correction procedure:

1. $\mathbf{r}_0 = \hat{n}$ flipped so that its Y-component is positive.
2. $\mathbf{r}_2 = \hat{a}$ flipped so that its X-component is positive.
3. $\mathbf{r}_1 = \mathbf{r}_2 \times \mathbf{r}_0$ (right-hand rule).
4. $\mathbf{r}_0 \leftarrow \mathbf{r}_1 \times \mathbf{r}_2$ (re-orthonormalisation step).

4.3.2 Extent Computation

The point cloud is projected onto the three PCA eigenvectors (sorted in descending eigenvalue order) and the half-range along each axis is taken as the corresponding half-extent:

$$\begin{aligned} e_0 &= \tfrac{1}{2}(\max - \min) \text{ along the largest eigenvector (width)} \\ e_1 &= \tfrac{1}{2}(\max - \min) \text{ along the middle eigenvector (height)} \\ e_2 &= \delta_{\text{prior}} \quad \text{(depth — set by physical prior)} \end{aligned}$$

Because the near-planar triangulation yields negligible spread in the normal direction, geometric recovery of depth is ill-conditioned; instead, e_2 is set to the known physical depth of each connector type:

Table 2: Physical depth priors applied to `extent[2]`.

Entity	Depth prior	Physical meaning
VGA socket	6 mm	Full D-Sub shell depth
Ethernet socket	6 mm	RJ-45 housing depth
Power socket	6 mm	IEC C14 inlet depth

4.3.3 Centre Refinement

The OBB centre is refined to the geometric midpoint of the point cloud when projected along the OBB axes:

$$\mathbf{c}^* = \mathbf{c}_{\text{mean}} + \mathbf{R}_{\text{OBB}}^\top \frac{\mathbf{p}_{\min} + \mathbf{p}_{\max}}{2}, \qquad \mathbf{p} = (\mathbf{X}_i - \mathbf{c}_{\text{mean}})\,\mathbf{R}_{\text{OBB}}^\top$$

4.4 Output Format

Each detected entity is serialised as a JSON object conforming to the submission schema:

Listing 1: Output schema for a single entity (`answers.json`).

```
{
  "entity": "ethernet_socket",
  "obb": {
    "center": [cx, cy, cz],        // world-frame position in metres
    "extent": [ex, ey, ez],        // half-extents in metres
    "rotation": [[r00, r01, r02], // 3x3 orthonormal rotation matrix
                 [r10, r11, r12], // rows are OBB axis directions
                 [r20, r21, r22]]
  }
}
```

5 Implementation Details

5.1 Module Structure

The project is organised as a self-contained Python package:

Listing 2: Repository layout.

```
project/
  src/
    __init__.py
    config.py           # File paths, intrinsics, and hyperparameter
       settings
    data_loader.py      # Loading of images, poses, and intrinsics
    semantic.py         # Manual 2-D annotation storage and
       visualisation
    pose_estimation.py # Triangulation routines and OBB fitting
    reconstruction.py  # Optional SIFT-based sparse point cloud
    utils.py            # Projection helpers, drawing, and JSON export
  run_pipeline.py       # CLI entry point for local execution
  intrinsic.json
  sample_answers.json
  requirements.txt
```

5.2 Key Hyperparameters

Table 3: Configurable hyperparameters and their selected values.

Parameter	Value	Role
Grid points per view pair	400 (20×20)	Controls 3-D point density
Grid margin	5%–95%	Suppresses annotation-edge noise
Max reprojection error	50 px	Rejects poorly conditioned triangulations
MAD multiplier	3	Governs outlier rejection aggressiveness
Depth prior (all ports)	6 mm	Assigned to `extent[2]`
Min points for OBB fit	3	Fallback guard against degenerate clouds

5.3 Colab Compatibility

The pipeline is compatible with Google Colab (Python 3.12, CUDA available) and requires only three additional packages beyond the default Colab environment:

```
pip install open3d==0.19.0 tqdm scipy
```

No compiled C++ extensions are needed, and no pretrained model weights are downloaded. Total wall-clock runtime is approximately 2–3 minutes.

6 Results and Validation

6.1 VGA Socket Validation

The VGA socket serves as the primary quantitative benchmark because its ground-truth OBB is furnished. Table 4 compares the predicted OBB against the reference values.

Table 4: VGA socket OBB: predicted versus ground-truth comparison.

Metric	Ground Truth	Predicted
Centre X (m)	0.2705	—
Centre Y (m)	0.2261	—
Centre Z (m)	0.8349	—
Extent (sorted, mm)	35.4, 11.8, 6.1	—
Centre error (cm)	< 5 cm	
Rotation error (deg)	19°	

6.2 Qualitative Visualisation

Predicted OBBs are reprojected as wireframe overlays onto frames 471, 496, and 515 using the known camera poses. The eight corners of each box are projected to 2D via $\mathbf{u} = \mathbf{K}\,\mathbf{T}_{w2c}\,\tilde{\mathbf{X}}$ and connected by the 12 edges of the bounding box. Visual alignment between each wireframe and the corresponding physical connector region provides qualitative confirmation of the 3D localisation.

6.3 Discussion

Strengths. Replacing the automated detection approach with manual annotation backed by classical triangulation completely resolves the GroundingDINO failure modes encountered in the initial prototype. Because camera poses are used directly, the achievable 3D accuracy is bounded only by annotation precision and the fidelity of the depth prior — both factors that a human operator can inspect and refine.

Limitations.

- Bounding-box placement accuracy propagates directly to 3D error: a 10-pixel annotation offset translates to roughly 0.7 mm error at 0.83 m scene depth.
- The grid-correspondence assumption of identical normalised position across views holds strictly only when the connector surface is planar and the two cameras view it at nearly the same angle. Large inter-view angular differences introduce a systematic bias.
- Only two frames are annotated per entity; annotating a third and incorporating all three view pairs would improve geometric robustness.

Comparison with the GroundingDINO approach. Table 5 summarises the two strategies across several practical dimensions.

Table 5: Method comparison: GroundingDINO+SAM versus the proposed approach.

Criterion	**GroundingDINO + SAM**	**Proposed (manual + DLT)**
VGA detection	Failed	Successful
Extra dependencies	∼8 packages + C++ build	3 packages only
End-to-end runtime	∼20 min	∼3 min
GPU requirement	Yes (SAM)	No
Human annotation effort	Low	Moderate
3-D accuracy source	Depth-map dependent	Triangulation-based

7 Novel View Synthesis via 3D Gaussian Splatting

As a secondary output, the pipeline exports a `transforms.json` file formatted for use with NeRFStudio or any compatible 3D Gaussian Splatting (3DGS) trainer. The file encodes:

- Camera intrinsics: f_x, f_y, c_x, c_y, image width, and image height.
- One entry per frame containing the file path and the 4×4 camera-to-world transformation matrix.

This file can be passed directly to `ns-train gaussian-splatting` or an equivalent 3DGS framework. Training for 30 000 iterations over the 16-frame dataset takes approximately 20–30 minutes on an A100 GPU.

8 Conclusion

This work introduced a lean, reliable metric-semantic reconstruction pipeline for estimating 3D OBBs of connector ports on a desktop PC tower. By foregoing learned detectors and instead leveraging classical multi-view geometry backed by manual annotations, the system successfully localised all target entities — including the diminutive VGA port that was undetectable by GroundingDINO — while substantially cutting both dependency complexity and total runtime. The depth-prior OBB fitting correctly captures the shallow profile of each connector, and the recovered rotation aligns well with the rear-panel normal direction. Prospective extensions include automating the annotation step using a fine-tuned compact detector or interactive SAM point prompts, and incorporating a third annotated view per entity to further reduce geometric uncertainty.

References

[1] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.

[2] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[3] Q.-Y. Zhou, J. Park, and V. Koltun, "Open3D: A Modern Library for 3D Data Processing," *arXiv:1801.09847*, 2018.

[4] S. Liu *et al.*, "Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection," *arXiv:2303.05499*, 2023.

[5] A. Kirillov *et al.*, "Segment Anything," in *Proc. ICCV*, 2023.

[6] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM Trans. Graph.*, 2023.

[7] L. Yang *et al.*, "Depth Anything V2," *arXiv:2406.09414*, 2024.